

The **engvis** UI Shell:

Menu Bar, Workflow Panels, Status Bar, and Mesh I/O for Implicit Surface Visualization

engvis

September 2, 2026

Abstract

This note documents the user-interface layer that wraps the **engvis** implicit surface viewer. Three things are described: the four-region **egui** layout (top menu, left numbered workflow, right detail panel, bottom status bar); a small triangle-mesh I/O layer covering OBJ / STL / PLY (built on **tobj**, **stl_io** and **ply-rs-bw**); and an extended catalogue of built-in implicit surfaces — sphere, torus, gyroid, Schwarz P, Schwarz D, Schoen IWP, Neovius — together with arbitrary user-typed expressions parsed via **fidget-rhai**. The accompanying source files are **src/main.rs** and **src/mesh_io.rs**.

1 Goals and Scope

The viewer must let a user, without leaving the window:

1. pick or type an implicit expression $f(x, y, z)$,
2. polygonise it with one of two backends (Marching Cubes 33 or Manifold Dual Contouring),
3. adjust the display (surface, points, edges, opacity, overlays),
4. read off topology metrics $V, E, F, \chi = V - E + F$ and watertightness,
5. import and export triangle meshes in OBJ, STL, PLY (and load glTF for textured scenes).

The principle followed throughout is *do not reinvent*: all parsing, dialog and IO work is delegated to mature crates (Table 1).

Table 1: External crates used by the UI layer.

Concern	Crate	Role
Immediate-mode GUI	egui 0.33	widgets and layout
File dialog	rfd 0.15	native open/save
OBJ reader	tobj 4.0	mesh import
STL read+write	stl_io 0.8	binary STL IO
PLY reader	ply-rs-bw 2.0	ASCII / LE / BE
Expression $\rightarrow$ Tree	fidget-rhai 0.4.3	string $\rightarrow$ Tree
Topology	engvis-core	V, E, F, χ

2 Window Layout

The window is partitioned into four `egui` panels plus a central 3D viewport, as shown in Figure 1.

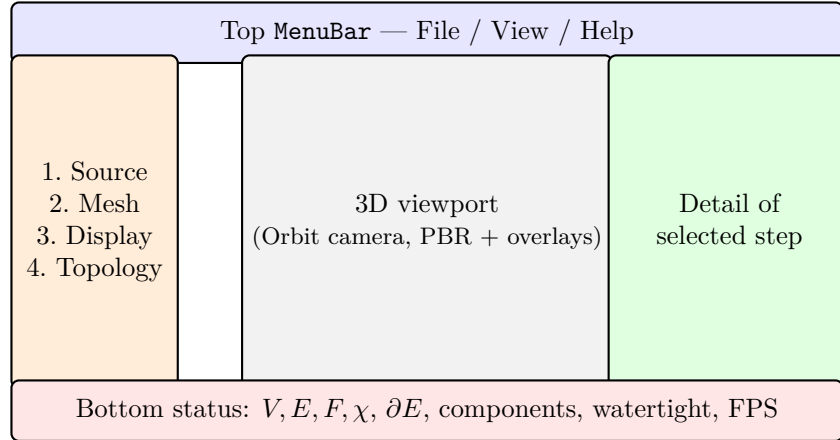


Figure 1: Four-panel `egui` layout. The left panel is the contract (“what to do”, numbered); the right panel is the implementation (“how”). Status bar reflects the topology of the currently shown mesh.

2.1 Top menu bar

Constructed with `egui::TopBottomPanel::top` and the new 0.33 `MenuBar` container. Three top-level menus:

File *Open mesh...* (OBJ / STL / PLY / glTF), *Save current mesh as {OBJ,STL,PLY}...*, *Quit*. File pickers are native via `rfd::FileDialog`; the picked path is written to `App::pending_load` / `App::pending_save` and serviced at the start of the next `ui()` call so that file I/O never happens inside an open menu.

View

toggles for surface, grid, point overlay, bounding wireframe, MS boundary loops.

Help

one-line description and the custom-expression syntax.

2.2 Left workflow panel (numbered, trait-style)

A simple `enum Step { Source, Mesh, Display, Topology }` governs which detail page the right panel renders. The numbered labels are rendered as `ui.selectable_labels` so that the panel acts like a four-row tab strip.

2.3 Right detail panel

A match on `self.current_step` dispatches to one of four small UI methods. Each method writes back into `App` fields and sets `self.needs_remesh = true` when it changes something that requires re-polygonisation.

2.4 Bottom status bar

A horizontal strip showing the topology of the most recent build:

$$V, E, F, \chi = V - E + F, \# \partial E, \# \text{components}, \text{watertight}.$$

This information is produced by `engvis_core::topology::compute_topology` (an existing half-edge analyser) and cached in `App::last_topology`.

2.5 Per-frame data flow

Figure 2 shows the order of operations inside `App::ui`.

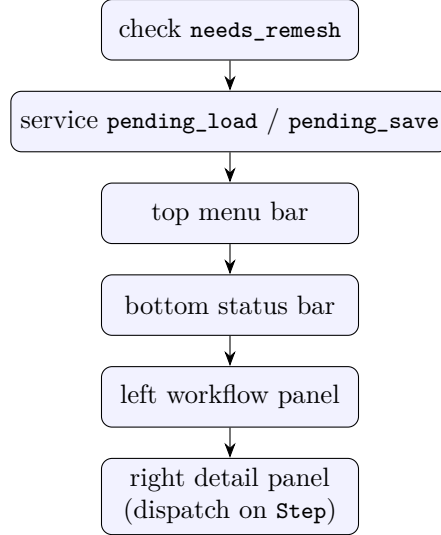


Figure 2: Per-frame UI sequence inside `App::ui`. Pending file actions are absorbed at the top so that a mesh swap is reflected in every panel below in the same frame.

3 Implicit Surface Library

Each built-in surface is a function returning a Fidget **Tree**. Trees are then handed to either Dual Contouring (`fidget-mesh`) or our Marching Cubes 33 implementation.

3.1 Closed-form definitions

Let $(x, y, z) \in \mathbb{R}^3$, $k > 0$ a period scale. We polygonise $\{f = 0\}$ on $\Omega = [-1, 1]^3$.

$$\text{Sphere} \quad \sqrt{x^2 + y^2 + z^2} - 0.8 = 0 \quad (1)$$

$$\text{Torus} \quad (\sqrt{x^2 + y^2} - R)^2 + z^2 - r^2 = 0, \quad R=0.6, \quad r=0.2 \quad (2)$$

$$\text{Gyroid} \quad \sin(kx) \cos(ky) + \sin(ky) \cos(kz) + \sin(kz) \cos(kx) = 0 \quad (3)$$

$$\text{Schwarz P} \quad \cos(kx) + \cos(ky) + \cos(kz) = 0 \quad (4)$$

$$\begin{aligned} \text{Schwarz D} \quad & \sin(kx) \sin(ky) \sin(kz) + \sin(kx) \cos(ky) \cos(kz) \\ & + \cos(kx) \sin(ky) \cos(kz) + \cos(kx) \cos(ky) \sin(kz) = 0 \end{aligned} \quad (5)$$

$$\begin{aligned} \text{Schoen IWP} \quad & 2[\cos(kx) \cos(ky) + \cos(ky) \cos(kz) + \cos(kz) \cos(kx)] \\ & - [\cos(2kx) + \cos(2ky) + \cos(2kz)] = 0 \end{aligned} \quad (6)$$

$$\text{Neovius} \quad 3[\cos(kx) + \cos(ky) + \cos(kz)] + 4 \cos(kx) \cos(ky) \cos(kz) = 0 \quad (7)$$

The Triply-Periodic Minimal Surfaces (TPMS) all use $k = 3$. The gyroid uses $k = 4$ to fit a few periods inside Ω . The variant *gyroid-sphere* reuses the gyroid expression and clips the resulting open patch to the unit ball post-mesh — see the existing *gyroid-sphere-clip* document.

3.2 User-typed expressions via `fidget-rhai`

`fidget-rhai::engine()` returns a Rhai engine pre-loaded with:

- `x`, `y`, `z` as **Tree** primitives (not numbers);
- all elementary operators (`+` `-` `*` `/` `)` overloaded to build new **Tree** nodes rather than evaluate;
- `sin`, `cos`, `sqrt`, `abs`, `exp`, `log`, ... as tree-producing free functions.

The user types e.g.

```
sin(4*x)*cos(4*y)+ sin(4*y)*cos(4*z)+ sin(4*z)*cos(4*x)
```

into the right-panel `TextEdit::multiline` widget. We then call:

```
let engine = fidget_rhai::engine();
let tree: fidget_core::context::Tree = engine.eval(src)?;
```

On error the message is shown in red below the editor and the surface build is skipped (the status bar shows “build failed”).

4 Mesh I/O Layer (`src/mesh_io.rs`)

4.1 Format dispatch

`MeshFormat::from_path` inspects the file extension and the two top-level functions `load_mesh` / `save_mesh` dispatch (Figure 3).

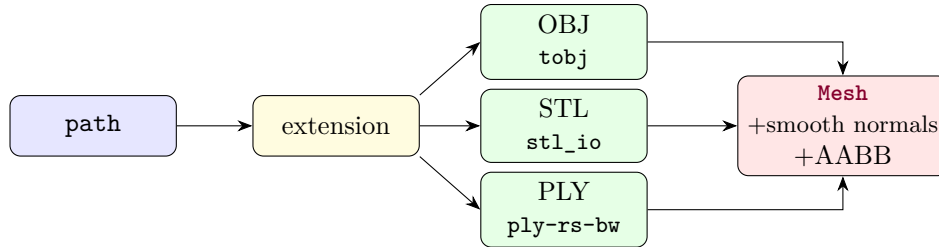


Figure 3: Format dispatch in `mesh_io`. Whatever the source, positions and indices are normalised and a single helper computes area-weighted vertex normals plus an AABB.

4.2 Smooth-normal helper

Imported meshes typically carry no normals (STL stores per-face normals only; OBJ frequently omits `vn`). We compute area-weighted vertex normals once during construction:

$$\mathbf{n}_v = \text{normalize} \sum_{T \ni v} (\mathbf{p}_1 - \mathbf{p}_0) \times (\mathbf{p}_2 - \mathbf{p}_0).$$

4.3 Writers

- **OBJ**: a ~ 20 -line writer emitting `v / vn / f a//a b//b c//c`. No external writer crate is needed; the format is trivially small.
- **STL**: `stl_io::write_stl` with a freshly recomputed face normal per triangle.
- **PLY**: a ~ 20 -line ASCII PLY writer. The typed-element API of `ply-rs-bw` is overkill for a one-shot export.

5 State Diagram of App

The user-visible state of the app is captured by a handful of fields; Figure 4 shows the transitions that trigger a remesh.

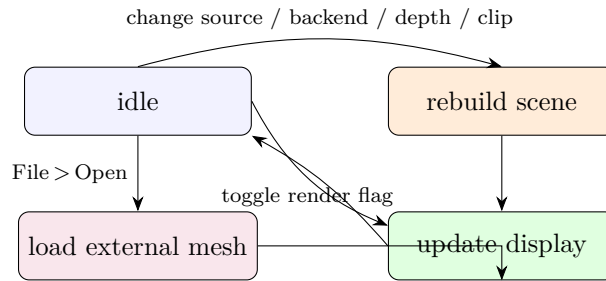


Figure 4: Coarse state diagram. Source, backend, depth, MS-loops and clip toggles all flip `needs_remesh`; pure display toggles (grid, points, opacity, edge color) update the next frame without a remesh.

6 CJK Font Fallback in the egui Backend

The UI text is drawn by `egui`, whose built-in font (`Ubuntu-Light`) contains no CJK glyphs; every Chinese character would render as an empty box (“■”). The fix lives in `crates/engvis-renderer/src/egui_integration.rs` and runs inside `EguiContext::new`, i.e. once per application start, before the first frame is laid out.

6.1 Fallback strategy

Rather than embedding a large CJK font into the binary, the renderer scans a prioritised list of system font paths (Table 2) and installs the first existing file as an *additional* font in both `egui` font families (`Proportional` and `Monospace`). `egui` resolves each glyph against the family list in order, so Latin text keeps the built-in font while CJK code points fall through to the system font:

$$g \mapsto \begin{cases} \text{built-in glyph,} & g \in \text{Latin / symbols,} \\ \text{system CJK glyph,} & \text{otherwise.} \end{cases} \quad (8)$$

7 Build Notes

Dependency conflict. At the time of writing, `facet-path 0.44.5` declares an upstream dependency on `facet-core "0.45"` while sibling crates in the `fidget` stack still pin `facet 0.44.x`, which itself uses `facet-core 0.44.x`. This produces a duplicate `facet-core` in the dependency graph and the trait `Facet<'_>` ceases to be implemented for `Tree`. We pin via

Table 2: System CJK font candidates, in priority order. For .ttc collections the first face (index 0) is used.

Platform	Path	Face
macOS	Hiragino Sans GB.ttc	Hiragino Sans GB W3
macOS	PingFang.ttc	PingFang
macOS	STHeiti Medium.ttc	Heiti
macOS	Supplemental/Arial Unicode.ttf	Arial Unicode
Windows	C:\Windows\Fonts\msyh.ttc	Microsoft YaHei
Linux	NotoSansCJK-Regular.ttc	Noto Sans CJK
Linux	wqy-microhei.ttc	WenQuanYi Micro Hei

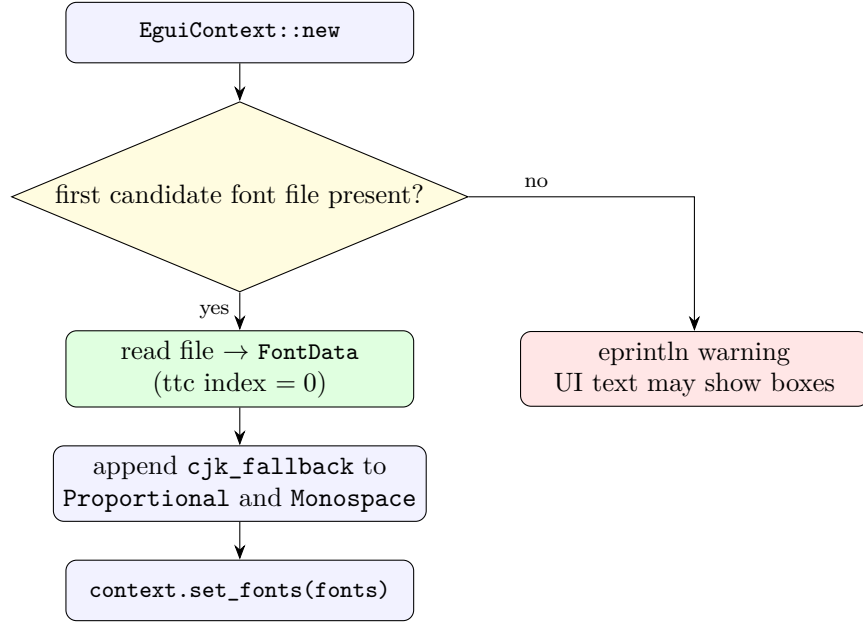


Figure 5: Font installation at egui context creation. The fallback is appended *last* in each family, so it only serves glyphs missing from the built-in fonts; missing fonts degrade to a console warning.

```
cargo update -p facet-path --precise 0.44.4
```

This is recorded in `Cargo.lock`; no source code change is needed.

Verification.

```

cargo build -p engvis      # clean build, no warnings from this crate
cargo run -p engvis        # window opens; default surface gyroid-sphere

```

On startup `EguiContext::new` installs the CJK fallback font (Figure 5); if no candidate file exists, a warning: `no CJK system font found` line is printed on `stderr`. Status-bar readouts are produced by `compute_topology` immediately after each successful mesh build, ensuring the displayed V, E, F, χ correspond to exactly the geometry on screen.